

Lista de exercícios — Listas Encadeadas

Instruções:

- Use a estrutura base fornecida.
- Nos exercícios de implementação, implemente as funções conforme as regras indicadas.
- Nos exercícios de análise de código: (i) localize o problema, (ii) explique a consequência, (iii) corrija com a menor alteração possível.

Lista simplesmente encadeada

Estrutura base 1 (próximos 6 exercícios):

Catálogo de livros (código identifica o livro; preço é o valor do livro)

```
typedef struct No {  
    int codigo;  
    float preco;  
    struct No* proxNo;  
} No;  
  
typedef struct {  
    No* inicio;  
    int tamanho;  
} Lista;
```

Remoção por critério (remover todos acima de um preço)

1. Implemente:

```
void removerAcimaDoPreco(Lista* lista, float limite);
```

Regras:

- Remova todos os nós cujo `preco > limite`.
- Trate corretamente a remoção do primeiro nó.
- Atualize tamanho a cada remoção.

Gabarito:

```

void removerAcimaDoPreco(Lista* lista, float limite) {
    if (lista == NULL) return;

    No* anterior = NULL;
    No* atual = lista->inicio;

    while (atual != NULL) {
        if (atual->preco > limite) {
            No* apagar = atual;
            atual = atual->proxNo; // guarda o próximo antes do free

            if (anterior == NULL) {
                // removendo o primeiro nó
                lista->inicio = apagar->proxNo;
            } else {
                anterior->proxNo = apagar->proxNo;
            }

            free(apagar);
            lista->tamanho--;
            // anterior NÃO avança quando remove
        } else {
            anterior = atual;
            atual = atual->proxNo;
        }
    }
}

```

Busca do maior preço (retornar ponteiro para o nó)

2. Implemente:

```
No* encontrarMaiorPreco(Lista* lista);
```

Regras:

- Retorne um ponteiro para o nó com maior preço.
- Se a lista estiver vazia, retorne NULL.

Gabarito:

```

No* encontrarMaiorPreco(Lista* lista) {
    if (lista == NULL || lista->inicio == NULL) return NULL;

    No* maior = lista->inicio;
    No* atual = lista->inicio->proxNo;

    while (atual != NULL) {
        if (atual->preco > maior->preco) {
            maior = atual;
        }
        atual = atual->proxNo;
    }
}

```

```

        return maior;
    }

```

Remoção de nós duplicados por código (mantendo a primeira ocorrência)

3. Implemente:

```
void removerDuplicadosPorCodigo(Lista* lista);
```

Regras:

- Remova nós duplicados considerando duplicado quando código for igual.
- Mantenha a **primeira** ocorrência de cada código.
- Atualize tamanho a cada remoção.

Gabarito:

```

void removerDuplicadosPorCodigo(Lista* lista) {
    if (lista == NULL) return;

    No* atual = lista->inicio;

    while (atual != NULL) {
        No* anterior = atual;
        No* comparar = atual->proxNo;

        while (comparar != NULL) {
            if (comparar->codigo == atual->codigo) {
                No* apagar = comparar;
                comparar = comparar->proxNo;

                anterior->proxNo = apagar->proxNo;
                free(apagar);
                lista->tamanho--;
                // anterior NÃO avança
            } else {
                anterior = comparar;
                comparar = comparar->proxNo;
            }
        }
        atual = atual->proxNo;
    }
}

```

4. Analise o código e faça:

1. Identifique **dois problemas** no trecho.
2. Explique a consequência de cada problema.
3. Corrija com a menor alteração possível, mantendo o estilo da aula.

```

void removerAcimaDoPreco(Lista* lista, float limite) {
    No* anterior = NULL;
    No* atual = lista->inicio;

    while (atual != NULL) {
        if (atual->preco > limite) {

            // (1) tentativa de remover o nó atual
            anterior->proxNo = atual->proxNo;

            // (2) libera e tenta avançar
            free(atual);
            atual = atual->proxNo;

            lista->tamanho--;
        } else {
            anterior = atual;
            atual = atual->proxNo;
        }
    }
}

```

Gabarito:

- Problema 1:** quando remove o primeiro nó, `anterior == NULL` e o código faz `anterior->proxNo = ....` Isso acessa memória inválida. **Consequência:** pode causar **segmentation fault**.
- Problema 2:** após `free(atual)`, o código usa `atual->proxNo`. Isso é **use-after-free**. **Consequência:** comportamento indefinido (pode travar, pular nós, corromper a lista).

Correção mínima (mantendo o estilo):

```

void removerAcimaDoPreco(Lista* lista, float limite) {
    No* anterior = NULL;
    No* atual = lista->inicio;

    while (atual != NULL) {
        if (atual->preco > limite) {
            No* apagar = atual;
            atual = atual->proxNo; // guarda o próximo antes do free

            if (anterior == NULL) {
                lista->inicio = apagar->proxNo;
            } else {
                anterior->proxNo = apagar->proxNo;
            }

            free(apagar);
            lista->tamanho--;
        } else {
            anterior = atual;
        }
    }
}

```

```

        atual = atual->proxNo;
    }
}

```

5. Analise o código e faça:

1. Explique por que o código não garante remoção correta dos nós duplicados por código.
2. Aponte a linha (ou bloco) responsável pelo problema principal.
3. Apresente uma correção funcional, mantendo a ideia original.

```

void removerDuplicadosPorCodigo(Lista* lista) {
    No* atual = lista->inicio;

    while (atual != NULL) {
        No* anterior = atual;
        No* comparar = atual->proxNo;

        while (comparar != NULL) {
            if (comparar->codigo == atual->codigo) {
                free(comparar);
                lista->tamanho--;
                comparar = comparar->proxNo;
            } else {
                anterior = comparar;
                comparar = comparar->proxNo;
            }
        }
        atual = atual->proxNo;
    }
}

```

Gabarito:

1. Porque ao encontrar duplicado ele faz `free(comparar)` **sem ajustar** o ponteiro do nó anterior (`anterior->proxNo`). Além disso, depois do `free` ele tenta usar `comparar->proxNo`.
2. O problema principal está no bloco: `free(comparar); ... comparar = comparar->proxNo;`
3. Correção: guardar o nó a remover, avançar o `comparar`, ajustar `anterior->proxNo` e só então dar `free`.

Correção funcional (mesma ideia):

```

void removerDuplicadosPorCodigo(Lista* lista) {
    No* atual = lista->inicio;

    while (atual != NULL) {
        No* anterior = atual;
        No* comparar = atual->proxNo;

```

```

while (comparar != NULL) {
    if (comparar->codigo == atual->codigo) {
        No* apagar = comparar;
        comparar = comparar->proxNo;

        anterior->proxNo = apagar->proxNo;
        free(apagar);
        lista->tamanho--;
    } else {
        anterior = comparar;
        comparar = comparar->proxNo;
    }
}
atual = atual->proxNo;
}
}

```

6. Responda:

1. Ao remover um nó, por que é necessário guardar o próximo nó antes de liberar memória?
2. Dê um exemplo de sequência de chamadas em que um ponteiro retornado anteriormente pode se tornar inválido.
3. Em uma remoção múltipla, em que situação o ponteiro anterior não deve avançar?

Gabarito:

1. Porque após `free(no)` não é seguro acessar `no->proxNo` (comportamento indefinido). Guardar o próximo antes evita **use-after-free**.
2. Exemplo: `No* p = encontrarMaiorPreco(lista);` e depois executar `removerAcimaDoPreco(lista, ...)` removendo esse mesmo nó. O ponteiro `p` passa a apontar para memória já liberada.
3. Quando o nó `atual` é removido. Nesse caso, `anterior` deve permanecer (ele continua sendo o nó anterior do próximo candidato), para permitir remoções consecutivas corretamente.

7. Considere a lista abaixo (cada nó é (codigo, preco)):

(12, 18.5) → (7, 42.0) → (9, 18.5) → (12, 11.0) → (4, 55.0) → NULL

1. Após executar `removerAcimaDoPreco(lista, 20.0)`, como fica a lista?
2. Em cada remoção, indique quando o ponteiro anterior deve permanecer no mesmo nó.

Gabarito:

1. Remove os nós com preço > 20: remove (7, 42.0) e (4, 55.0). Resultado:

(12, 18.5) → (9, 18.5) → (12, 11.0) → NULL

2. `anterior` permanece no mesmo nó **sempre que ocorre remoção** (pois `atual` muda, mas o anterior do próximo candidato continua sendo o mesmo). Ele só avança quando `atual` não é removido.

8. Ao remover um nó do meio em uma lista duplamente encadeada, quais ponteiros precisam ser atualizados nos vizinhos do nó removido? Escreva as duas atribuições principais (sem código completo).

Gabarito:

- `removido->ant->prox = removido->prox;`
- `removido->prox->ant = removido->ant;`

9. Em uma lista duplamente encadeada circular, por que não existe ponteiro NULL na estrutura?

Gabarito: Porque a lista é circular: o próximo do último aponta para o primeiro e o anterior do primeiro aponta para o último. Assim, o encadeamento forma um ciclo e não existem extremidades com NULL.